```
iex> Examples.Modules.Compile.runtime()
This was compiled at 20:26:17.999151. It is 20:26:47.052395
```

And if you open `iex -S mix` again, you won't see the `"I'm at compile-time"` message. You'll see the message again when you change the module and Elixir recompiles it.

This gets to the heart of the confusion for many new Elixir developers. They will put a value inside of a module attribute (`@now` in our example) and assume that it updates every time they use it. Instead, this type of code is executed at compile time and the value doesn't change.

A great way to avoid a problem is to put everything in functions when you are starting out. Whether you are accessing the database, collecting a system configuration variable, or reading a file, do it in a named function.

You'll bump up against this more as you write Elixir code. The good news is that the language maintainers have put a lot of work into making it obvious where the compile-time boundaries are. So, issues about compilation have steadily decreased over time.

Wrapping Up

Elixir can be strange to look at when you first start out, but its surface area is fairly small and easy to learn. Putting it together into a meaningful application is where the fun begins.

Elixir comes with a common set of data types and operators. These largely behave as you would expect, but a few rough edges sometimes trip up new Elixir programmers. You'll be set up for success by simply knowing that these edges exist.

Modules and functions are the building blocks of an Elixir application. Modules are containers of functions that you have grouped together. There are a few keywords that make modules more enjoyable to work with. The most powerful is the `use` keyword, which offers a hook point for library authors to create seamless domain-specific languages.

Functions are conceptually simple, but quite a few syntaxes are available to use in different situations. You'll most commonly use named functions, but you'll work with anonymous functions as well. Plus, Elixir comes with powerful function overloading capabilities.

It's easy to get tripped up when going from an interpreted language to a compiled language. Elixir's compiler transforms your text files into the modules and functions that make up your application. Put all of your code into named